TITLE: “Book Recommendation Analysis using Library Data”

Branch: AIML **Section:** 2 **Batch No:** 15

Student Roll No’s : 24B11AI281, 24B11AI276, 24B11AI239, 24B11AI327

Student Name’s : N.S.S. Devi, M. Padmavathi, M.A.V. Supriya, P. Varshitha devi

Faculty Name : M. Pravalika

ABSTRACT

This project focuses on analysing library data and book recommendation datasets to understand user preferences and improve recommendation systems. The system uses data mining techniques such as classification, clustering, and association rule mining to extract meaningful patterns from data. The datasets are preprocessed through steps like data cleaning, integration, and transformation to ensure accuracy and consistency. Algorithms like J48, Random Tree, K-Means, and Apriori are applied to predict user interests and identify relationships between books and users. Classification techniques help in predicting whether a user will like a particular book, while clustering groups similar users or books based on their behaviour and features. Association rule mining is used to discover hidden relationships and patterns among different attributes. The results are visualized using graphs such as bar charts and scatter plots for better understanding and analysis. This system helps libraries and online platforms provide better and personalized recommendations, reduces manual effort, and enhances overall user experience.

1. INTRODUCTION

1.1 Introduction

The project “Book Recommendation Analysis using Library Data” is developed to analyse user reading behaviour and suggest relevant books. With the rapid growth of data, manual analysis is difficult and inefficient. This application uses data mining techniques to process large datasets and generate meaningful insights.

The system performs operations such as data loading, cleaning, integration, transformation, and analysis. It helps users by recommending books based on their interests and past behaviour

1.2 Problem Statement

In the current system, there is no proper method to provide personalized book recommendations. Users must manually search for books, which is time-consuming and inefficient. Handling large datasets manually leads to errors and difficulty in analysis. Therefore, a computerized system is required to process data automatically and generate accurate recommendations using data mining techniques.

1.3 Objectives of the Project

- To analyse book recommendation datasets
- To clean and preprocess the data
- To apply classification algorithms (J48, Random Tree)
- To perform clustering using K-Means algorithm
- To generate association rules using Apriori algorithm
- To identify user behavior patterns
- To improve recommendation accuracy

1.4 Scope of the Project

The system can be used in libraries, educational institutions, and online book platforms to recommend books based on user preferences.

It supports operations such as:

- Data analysis
- Pattern identification
- User grouping
- Recommendation generation

This system improves efficiency and helps users easily find suitable books.

2. SYSTEM OVERVIEW

2.1 Existing System

In the existing system, book recommendations are not personalized. Users search for books manually using catalogs or lists. Data is often maintained in spreadsheets or records, which makes analysis difficult and time-consuming.

2.2 Limitations of Existing System

- Manual errors may occur
- Data retrieval is difficult
- Time-consuming process
- No personalized recommendations
- Difficult to analyse large datasets

2.3 Proposed System

The proposed system uses data mining techniques to analyze book and library datasets and generate personalized recommendations.

It automates tasks such as:

- Data cleaning
- Data integration
- Data transformation
- Data analysis

The system uses Python and data mining algorithms to improve efficiency and accuracy.

2.4 Software Requirements

- Python
- Jupyter Notebook / Google Colab
- Libraries: Pandas, NumPy, Matplotlib, Scikit-learn
- Operating System: Windows

2.5 Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB or above
- Hard Disk: 10 GB free space
- Monitor and keyboard

3. TECHNOLOGIES USED

3.1 Python

Python is a high-level programming language widely used for data analysis and machine learning. It is easy to learn and has a simple syntax. In this project, Python is used for data preprocessing, analysis, and implementing algorithms. Its powerful libraries make it suitable for handling large datasets efficiently.

3.2 Pandas

Pandas is a Python library used for data manipulation and analysis. It provides data structures like Data Frames and Series. In this project, it is used for loading data, cleaning datasets, and performing transformations.

3.3 NumPy

NumPy is used for numerical computations and handling arrays. It helps in performing mathematical operations efficiently, especially on large datasets.

3.4 Matplotlib

Matplotlib is used for data visualization. It helps in creating graphs such as bar charts and scatter plots to understand data patterns clearly

3.5 Scikit-learn

Scikit-learn is a machine learning library used to implement algorithms like classification and clustering. It provides simple tools to build predictive models.

4. DATABASE DESIGN

4.1 Dataset Description

The project uses two datasets:

- Book Recommendation Dataset
- Library Dataset

These datasets contain information such as user IDs, book titles, genres, ratings, and reading history. The data is collected from different sources and combined for analysis. These datasets help in understanding user preferences and identifying patterns in reading behaviour.

4.2 Data Cleaning

Data cleaning is an important step to improve data quality. It involves:

- Removing null or missing values
- Removing duplicate records
- Correcting inconsistent data
- Formatting data properly

This step ensures that the dataset is accurate and ready for analysis.

4.3 Data Integration

Data integration involves combining multiple datasets into a single dataset. In this project, the book recommendation dataset and library dataset are merged to create a unified dataset. This helps in improving the quality of analysis and generating better insights.

4.4 Data Transformation

Data transformation converts raw data into a suitable format for analysis. It includes:

- Converting categorical data into numerical values
- Normalizing data
- Scaling values
- Feature selection

This step helps in improving the performance of machine learning algorithms.

4.5 Sample Data

Sample data represents the structure of the dataset used in the project. It includes fields like user ID, book title, rating, and genre.

5. SYSTEM MODULES

Module 1: Data Loading

This module is responsible for loading datasets into the system. It reads data from files such as CSV or Excel and stores it in data structures for further processing.

Module 2: Data Cleaning

This module removes errors and inconsistencies in the dataset. It ensures that the data is complete, accurate, and ready for analysis.

Module 3: Data Integration

This module combines multiple datasets into a single dataset. It helps in improving data quality and enables better analysis.

Module 4: Data Analysis

This module applies various data mining techniques such as classification and association rule mining to extract meaningful insights from the data.

Module 5: Clustering

This module groups similar users or books based on their characteristics. It helps in identifying patterns and improving recommendations.

6. IMPLEMENTATION

6.1 Classification

Classification is used to predict whether a user will like a particular book based on their previous behaviour and preferences. It is a supervised learning technique.

J48 Algorithm

J48 is a decision tree algorithm that splits the dataset based on attributes. It selects the best attribute using information gain and builds a tree structure. It is easy to understand and helps in making predictions effectively.

Random Tree

Random Tree is another decision tree-based algorithm that selects attributes randomly. It is faster and helps in reducing overfitting. It works well with large datasets and provides good prediction results.

6.2 Association Rule Mining

Apriori Algorithm

Apriori algorithm is used to find frequent item sets and generate association rules. It works based on support and confidence values.

Example:
Users who read Fiction books are likely to read Mystery books.

This helps in understanding relationships between different attributes and improves recommendation systems.

6.3 Clustering

Clustering is an unsupervised learning technique used to group similar data points.

K-Means Algorithm

K-Means divides data into a fixed number of clusters. Each cluster has a centroid, and data points are assigned based on distance from the centroid.

Steps Involved:

1. Select number of clusters (K)
2. Initialize centroids
3. Assign data points to nearest centroid
4. Update centroids
5. Repeat until convergence

Advantages:

- Easy to implement
- Fast and efficient
- Helps in identifying hidden patterns

Application in Project:

- Group similar users
- Improve recommendation accuracy

7. OUTPUT SCREENS

DataSet-1: Loading the dataset-1(**Book Recommendation dataset**)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Loading Datasets
user_df = pd.read_csv('user_interaction_with_nulls.csv')
print(user_df.head(5))
```

Output:

```
  UserID BookID BookTitle        Genre  UserRating ReadingFrequency  \
0   U500  B2192   Book_23  Non-Fiction         4.0              High
1   U501  B2247   Book_58           AI         NaN               Low
2   U502  B2195  Book_159           AI         4.0              High
3   U503  B2087   Book_47     Academic         1.0               NaN
4   U504  B2226   Book_43           ML         1.0               Low

  PreferredCategory  TimesViewed  TimesBorrowed  RecommendationScore  \
0       Electronics           18              6                 2.37
1       Electronics            5              2                 2.63
2             Maths           20              6                 4.51
3                AI           19              6                 2.48
4                DS           11              6                 2.71

  StudentDepartment AcademicYear
0               CSE     4th Year
1             CIVIL     3rd Year
2               CSE     3rd Year
3               EEE     2nd Year
4               EEE     4th Year
```

DataSet-2: Loading the dataset-2 (**Library dataset**)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
borrow_df = pd.read_csv('library_borrowing_with_nulls.csv')
print(borrow_df.head(5))
```

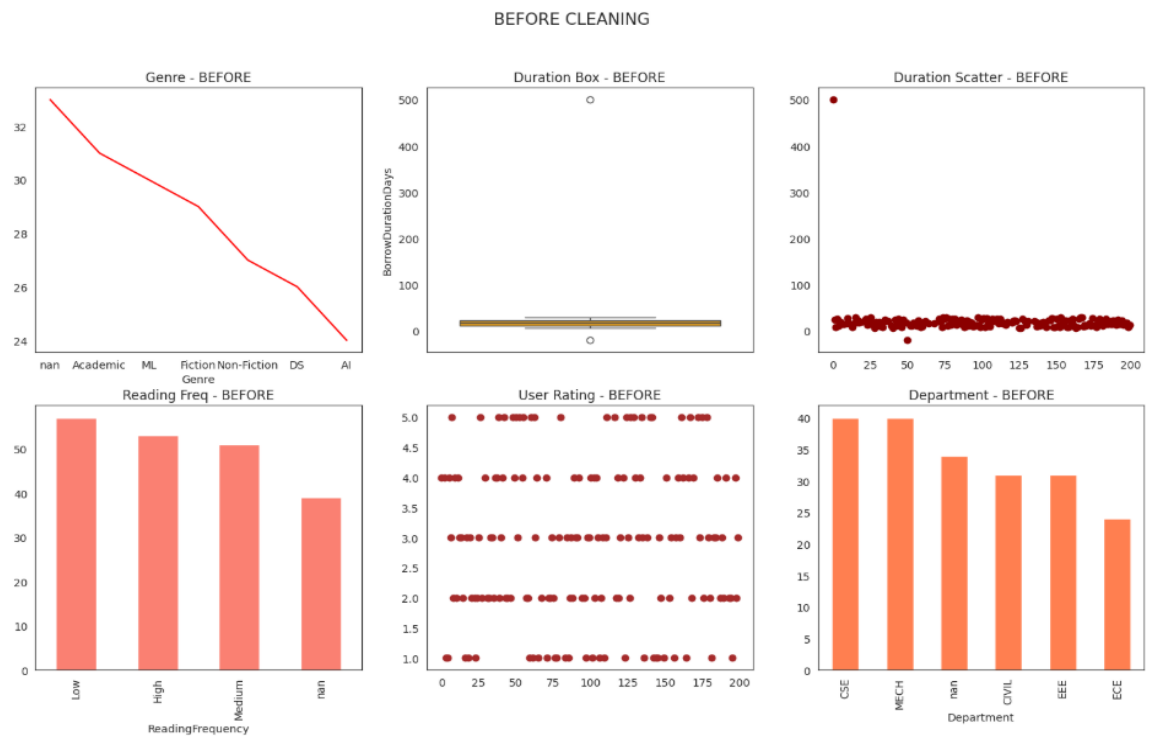
Output:

```
  StudentID BookID BookTitle      Author      Category  BorrowDate  \
0     S1000  B2166  Book_103   Author_57            ML  2024-01-26
1     S1001  B2078  Book_101  Author_100    Literature  2024-07-29
2     S1002  B2203   Book_27   Author_24  Data Science  2024-08-18
3     S1003  B2334  Book_124   Author_78   Electronics  2024-03-11
4     S1004  B2025  Book_163    Author_1            ML  2024-05-26

   ReturnDate  BorrowDurationDays  UserRating  TimesBorrowed Department  \
0         NaN                27.0           2              2        CSE
1  2024-06-25                24.0           4              1        CSE
2         NaN                 9.0           2              2      CIVIL
3  2024-04-03                27.0           3              6       MECH
4  2024-03-03                12.0           3              4       MECH

  AcademicYear
0     4th Year
1     3rd Year
2     1st Year
3     2nd Year
4     4th Year
```

Before Cleaning: Before cleaning the dataset-1 and dataset-2 ("**library data**" and **Book Recommendation dataset)**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("white")
# --- PRE-PREPARATION (Adding Nulls & Outliers for project demonstration) --
np.random.seed(42)
for col in ['UserRating', 'Genre', 'ReadingFrequency']:
    user_df.loc[user_df.sample(frac=0.1).index, col] = np.nan
for col in ['BorrowDurationDays', 'Department']:
    borrow_df.loc[borrow_df.sample(frac=0.1).index, col] = np.nan
# Adding Outliers
borrow_df.loc[0, 'BorrowDurationDays'] = 500  # High outlier
borrow_df.loc[50, 'BorrowDurationDays'] = -20 # Negative outlier

# --- BEFORE CLEANING  ---
plt.figure(figsize=(18, 10))
plt.suptitle("BEFORE CLEANING", fontsize=15)

# Row 1 (3 graphs)
plt.subplot(2, 3, 1)
user_df['Genre'].value_counts(dropna=False).plot(kind='line', color='red')
plt.title("Genre - BEFORE")

plt.subplot(2, 3, 2)
sns.boxplot(y=borrow_df['BorrowDurationDays'].dropna(), color='orange')
plt.title("Duration Box - BEFORE")
```

Output: To visualize the data correctly and accurately we have used Scatter Plots, Bar Charts.



Cleaning dataset-1:

```
# --- CLEANING SECTION dataset1 ---
user_df.columns = user_df.columns.str.strip()


# Fill Missing Values (Maintains row count)
#handling missing values using mode and mean
user_df['UserRating'] = user_df['UserRating'].fillna(user_df['UserRating'].mean())
user_df['Genre'] = user_df['Genre'].fillna(user_df['Genre'].mode()[0])
user_df['ReadingFrequency'] = user_df['ReadingFrequency'].fillna(user_df['ReadingFrequency'].mode()[0])
#removing duplicates
user_df = user_df.drop_duplicates()

# Text Normalization
user_df['ReadingFrequency'] = user_df['ReadingFrequency'].str.lower()
user_df['Genre'] = user_df['Genre'].str.lower()
borrow_df['Department'] = borrow_df['Department'].str.upper()
```

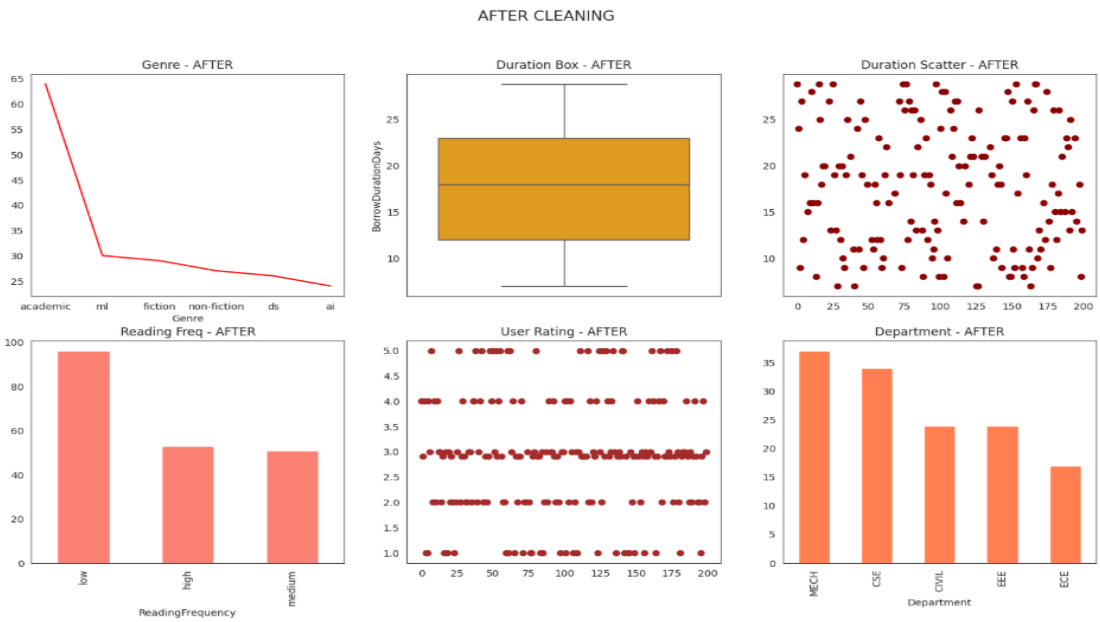
Cleaning dataset-2:

```
# --- CLEANING SECTION dataset2 ---
borrow_df.columns = borrow_df.columns.str.strip()
# Handle Outliers in Borrowing Data
borrow_df = borrow_df[borrow_df['BorrowDurationDays'] >= 0] # Remove negatives
q_hi = borrow_df['BorrowDurationDays'].quantile(0.95)
borrow_df.loc[borrow_df['BorrowDurationDays'] > q_hi, 'BorrowDurationDays'] = q_hi # Cap high outliers
#handling missing values using mean and mode
borrow_df['BorrowDurationDays'] = borrow_df['BorrowDurationDays'].fillna(borrow_df['BorrowDurationDays'].mean())

department_mode_series = borrow_df['Department'].mode()


# Text Normalization
borrow_df['Department'] = borrow_df['Department'].str.upper() # Corrected method call
```

After Cleaning: After cleaning the dataset-1 and dataset-2 ("**library dataset**" and **Book Recommendation dataset**)

```
# --- After CLEANING  ---
plt.figure(figsize=(18, 10))
plt.suptitle("AFTER CLEANING", fontsize=15)

# Row 1 (3 graphs)
plt.subplot(2, 3, 1)
user_df['Genre'].value_counts(dropna=False).plot(kind='line', color='red')
plt.title("Genre - AFTER")

plt.subplot(2, 3, 2)
sns.boxplot(y=borrow_df['BorrowDurationDays'].dropna(), color='orange')
plt.title("Duration Box - AFTER")

plt.subplot(2, 3, 3)
plt.scatter(borrow_df.index, borrow_df["BorrowDurationDays"], color='darkred')
plt.title("Duration Scatter - AFTER")

plt.subplot(2, 3, 4)
user_df['ReadingFrequency'].value_counts(dropna=False).plot(kind='bar', color='salmon')
plt.title("Reading Freq - AFTER")

plt.subplot(2, 3, 5)
plt.scatter(user_df.index, user_df["UserRating"], color='brown')
plt.title("User Rating - AFTER")

plt.subplot(2, 3, 6)
borrow_df['Department'].value_counts(dropna=True).plot(kind='bar', color='coral')
plt.title("Department - AFTER")
```

Output:



Integrating two datasets:

```
# --- INTEGRATION (Merge) ---
integrated_df = pd.merge(user_df, borrow_df, on='BookID', how='inner')

if len(integrated_df) != 200:
    integrated_df = integrated_df.sample(200, replace=True).reset_index(drop=True)
    print("Null values in integrated dataset")
    print(integrated_df[['ReturnDate','BorrowDate','Department']].isnull().sum())
```

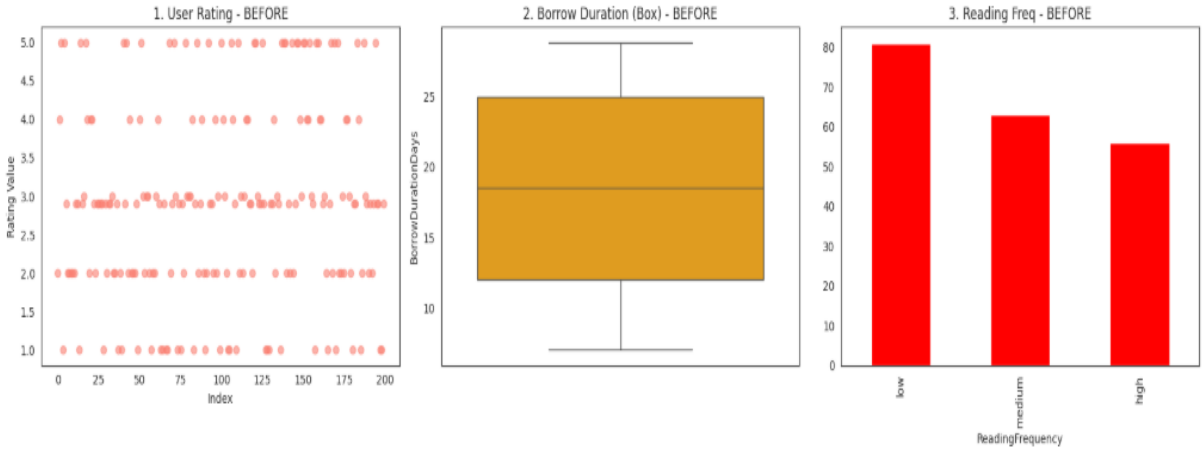
```
Null values in integrated dataset
ReturnDate     8
BorrowDate     3
Department    37
dtype: int64
```

Before cleaning integrated dataset:

```
# --- VISUALIZATION: BEFORE CLEANING (INTEGRATED DATASET)) ---
plt.figure(figsize=(18, 5))

plt.figure(figsize=(18, 5))

# Graph 1: Raw User Rating Scatter
plt.subplot(1, 3, 1)
plt.scatter(integrated_df.index, integrated_df['UserRating_x'], color='salmon', alpha=0.6)
plt.title("1. User Rating - BEFORE")
plt.xlabel("Index")
plt.ylabel("Rating Value")

# Graph 2: Raw Borrow Duration (Showing Outliers)
plt.subplot(1, 3, 2)
sns.boxplot(y=integrated_df['BorrowDurationDays'], color='orange')
plt.title("2. Borrow Duration (Box) - BEFORE")

# Graph 3: Reading Frequency Bar (Uncleaned)
plt.subplot(1, 3, 3)
integrated_df['ReadingFrequency'].value_counts(dropna=False).plot(kind='bar', color='red')
plt.title("3. Reading Freq - BEFORE")

plt.tight_layout()
plt.show()
```

Output:

```
<Figure size 1800x500 with 0 Axes>
```



Cleaning Integrated dataset:

```
#cleaning and handling missing value in integrated dataset
integrated_df['BorrowDate'] = integrated_df['BorrowDate'].fillna(integrated_df['BorrowDate'].mode()[0])
integrated_df['ReturnDate'] = integrated_df['ReturnDate'].fillna(integrated_df['ReturnDate'].mode()[0])
integrated_df['BorrowDurationDays'] = integrated_df['BorrowDurationDays'].fillna(integrated_df['BorrowDurationDays'].median())
print("After Cleaning Null Counts:")
integrated_df['Department'] = integrated_df['Department'].fillna(integrated_df['Department'].mode()[0])
integrated_df['Department'] = integrated_df['Department'].str.upper()
print("After Cleaning Null Counts:")
print(integrated_df[['ReturnDate','BorrowDate','Department']].isnull().sum())
```

```
After Cleaning Null Counts:
After Cleaning Null Counts:
ReturnDate    0
BorrowDate    0
Department    0
dtype: int64
```

Transformation of Integrated dataset:

```
# ---  TRANSFORMATION CODE ---
# Outliers handling
integrated_df['BorrowDurationDays'] = integrated_df['BorrowDurationDays'].clip(lower=0, upper=30)

# Transformation
integrated_df['RatingLevel'] = pd.cut(
    integrated_df['UserRating_x'],
    bins=[0, 2, 4, 5.1],
    labels=['low', 'medium', 'high']
)
integrated_df['BorrowDurationLevel'] = pd.cut(
    integrated_df['BorrowDurationDays'],
    bins=[0, 10, 20, 100],
    labels=['short', 'medium', 'long']
)
print(integrated_df.head(2))
integrated_df.to_csv('final.csv', index=False)
```

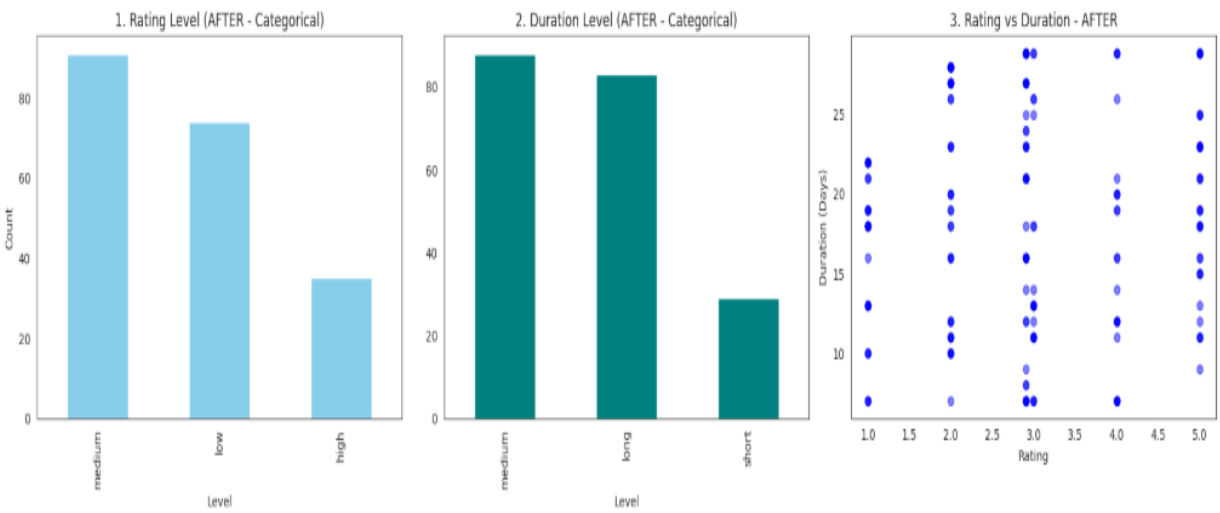
Output:

```
  UserID BookID BookTitle_x        Genre  UserRating_x ReadingFrequency  \
0   U693  B2263     Book_84           ml           2.0              low
1   U685  B2296    Book_150  non-fiction           4.0              low

  PreferredCategory  TimesViewed  TimesBorrowed_x  RecommendationScore  ...  \
0                DS          14                0                 1.26  ...
1                DS           1                1                 0.94  ...

      Category  BorrowDate  ReturnDate BorrowDurationDays UserRating_y  \
0  Mathematics  2024-05-07  2024-10-16               23.0            5
1           ML  2024-10-08  2024-11-26               12.0            2

  TimesBorrowed_y Department AcademicYear_y  RatingLevel  BorrowDurationLevel
0               6        EEE       3rd Year          low                 long
1               1       MECH       1st Year       medium               medium

[2 rows x 25 columns]
```

After Cleaning and Transformation of Integrated Dataset:

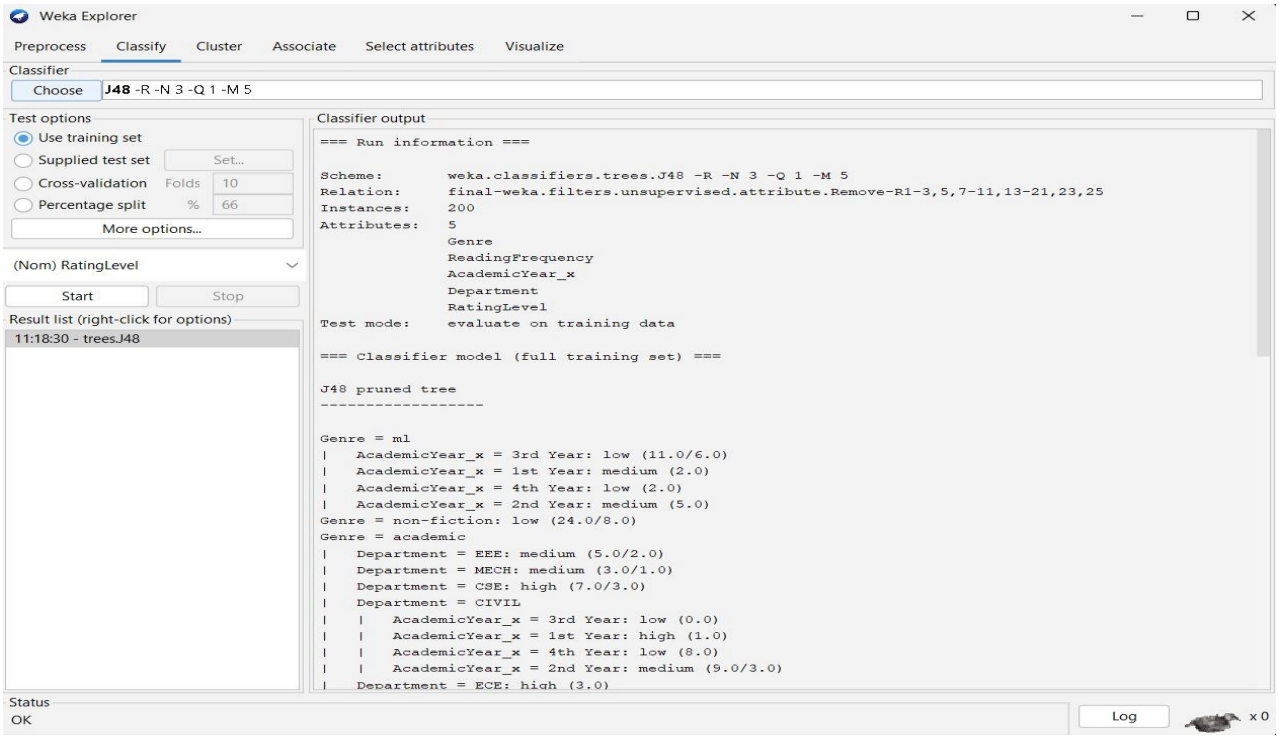
```python
# --- VISUALIZATION: AFTER PREPROCESSING(3 Graphs) ---
plt.figure(figsize=(18, 5))

# Graph 1: Rating Level (Transformed Category)
plt.subplot(1, 3, 1)
integrated_df['RatingLevel'].value_counts().plot(kind='bar', color='skyblue')
plt.title("1. Rating Level (AFTER - Categorical)")
plt.xlabel("Level")
plt.ylabel("Count")

# Graph 2: Borrow Duration Level (Transformed Category)
plt.subplot(1, 3, 2)
integrated_df['BorrowDurationLevel'].value_counts().plot(kind='bar', color='teal')
plt.title("2. Duration Level (AFTER - Categorical)")
plt.xlabel("Level")

# Graph 3: User Rating vs Duration (Cleaned Scatter)
plt.subplot(1, 3, 3)
plt.scatter(integrated_df['UserRating_x'], integrated_df['BorrowDurationDays'], color='blue', alpha=0.5)
plt.title("3. Rating vs Duration - AFTER")
plt.xlabel("Rating")
plt.ylabel("Duration (Days)")

plt.tight_layout()
plt.show()
```

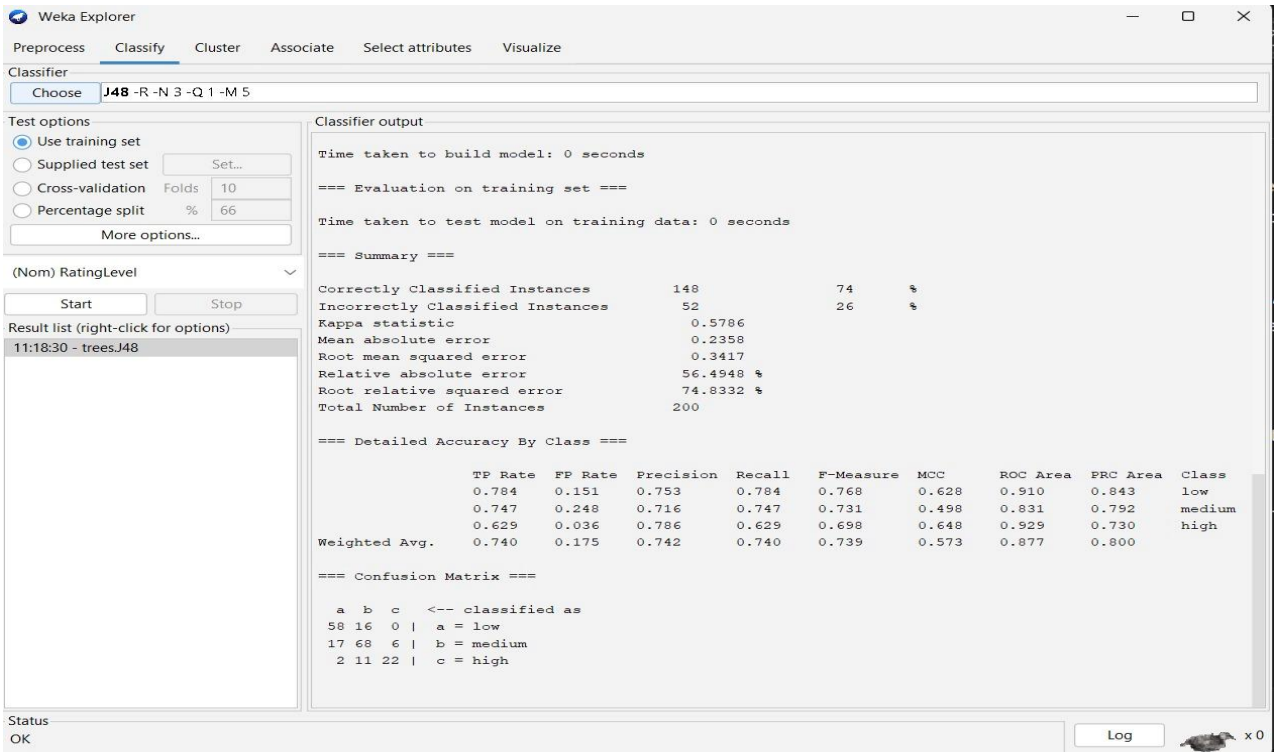
Output:

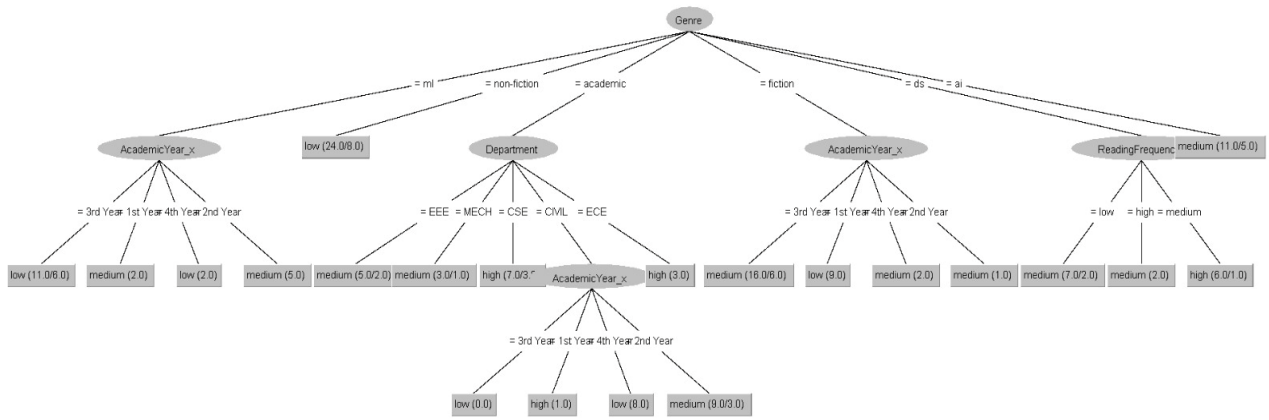


Methodology:

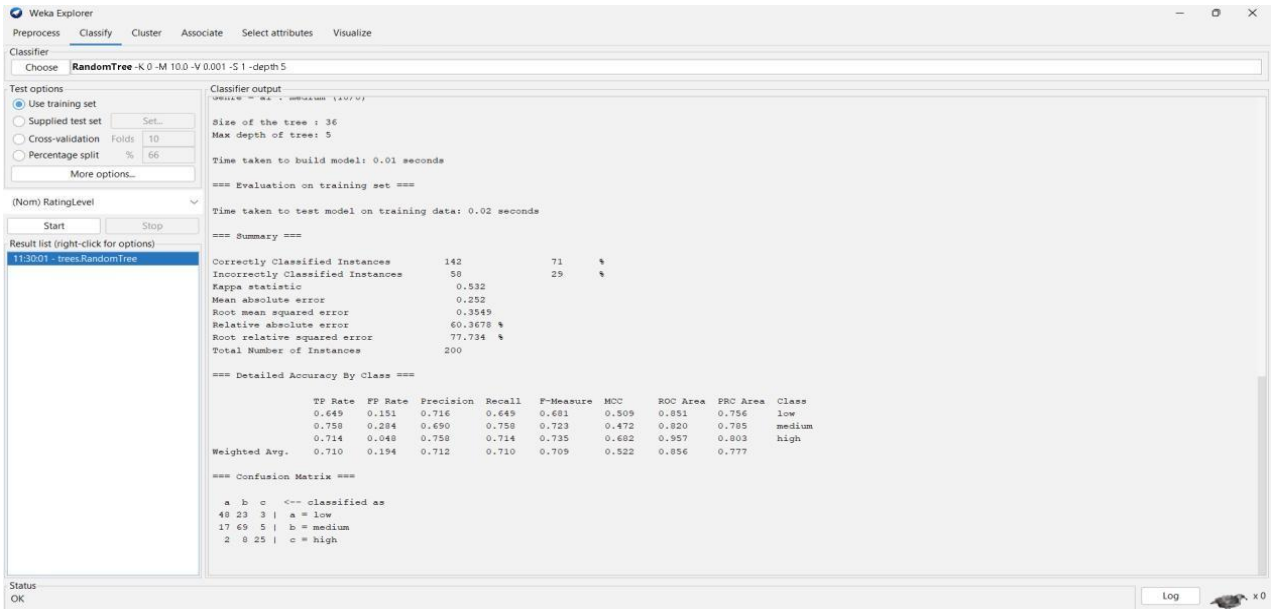
J48 Algorithm

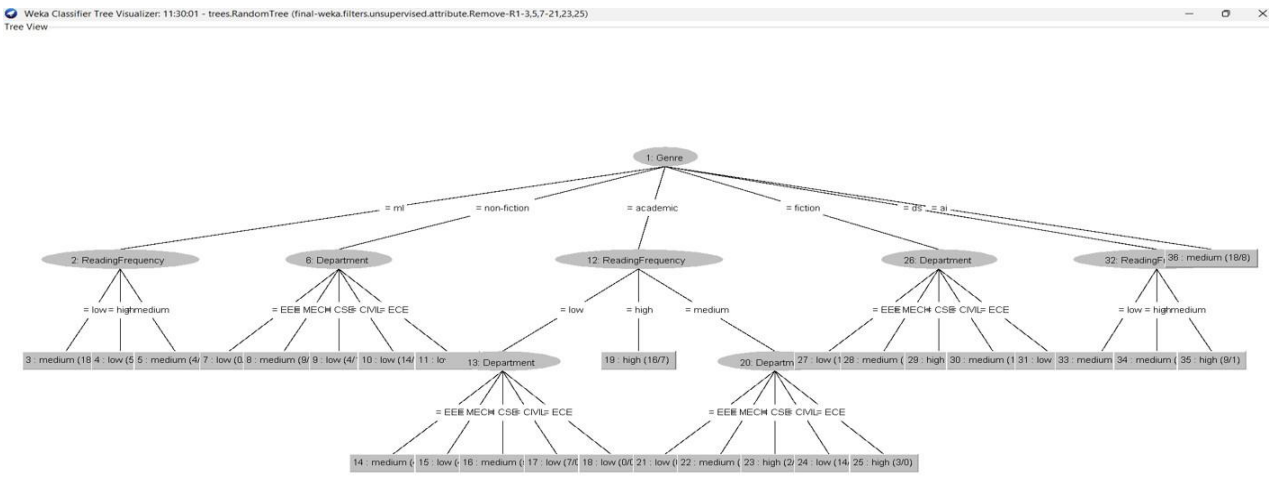





Genre
= ml
= non-fiction
= academic
= fiction
= ds
= ai
AcademicYear_x
low (24.0/8.0)
Department
AcademicYear_x
ReadingFrequenc
medium (11.0/5.0)
= 3rd Year
1st Year
4th Year
2nd Year
= EEE
= MECH
= CSE
= CIVIL
= ECE
= 3rd Year
1st Year
4th Year
2nd Year
= low
= high
= medium
low (11.0/6.0)
medium (2.0)
low (2.0)
medium (5.0)
medium (5.0/2.0)
medium (3.0/1.0)
high (7.0/3.
AcademicYear_x
high (3.0)
medium (16.0/6.0)
low (9.0)
medium (2.0)
medium (1.0)
medium (7.0/2.0)
medium (2.0)
high (6.0/1.0)
= 3rd Year
1st Year
4th Year
2nd Year
low (0.0)
high (1.0)
low (8.0)
medium (9.0/3.0)

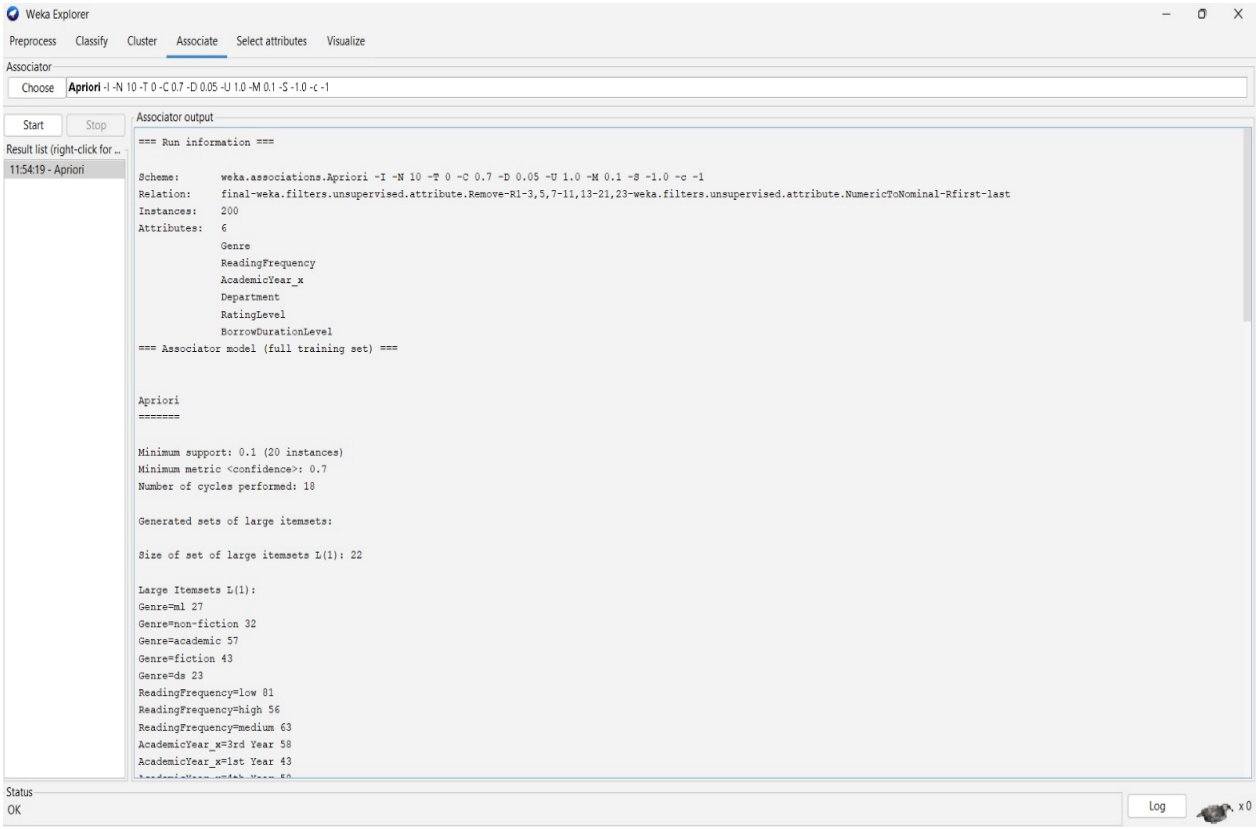
Random Tree:

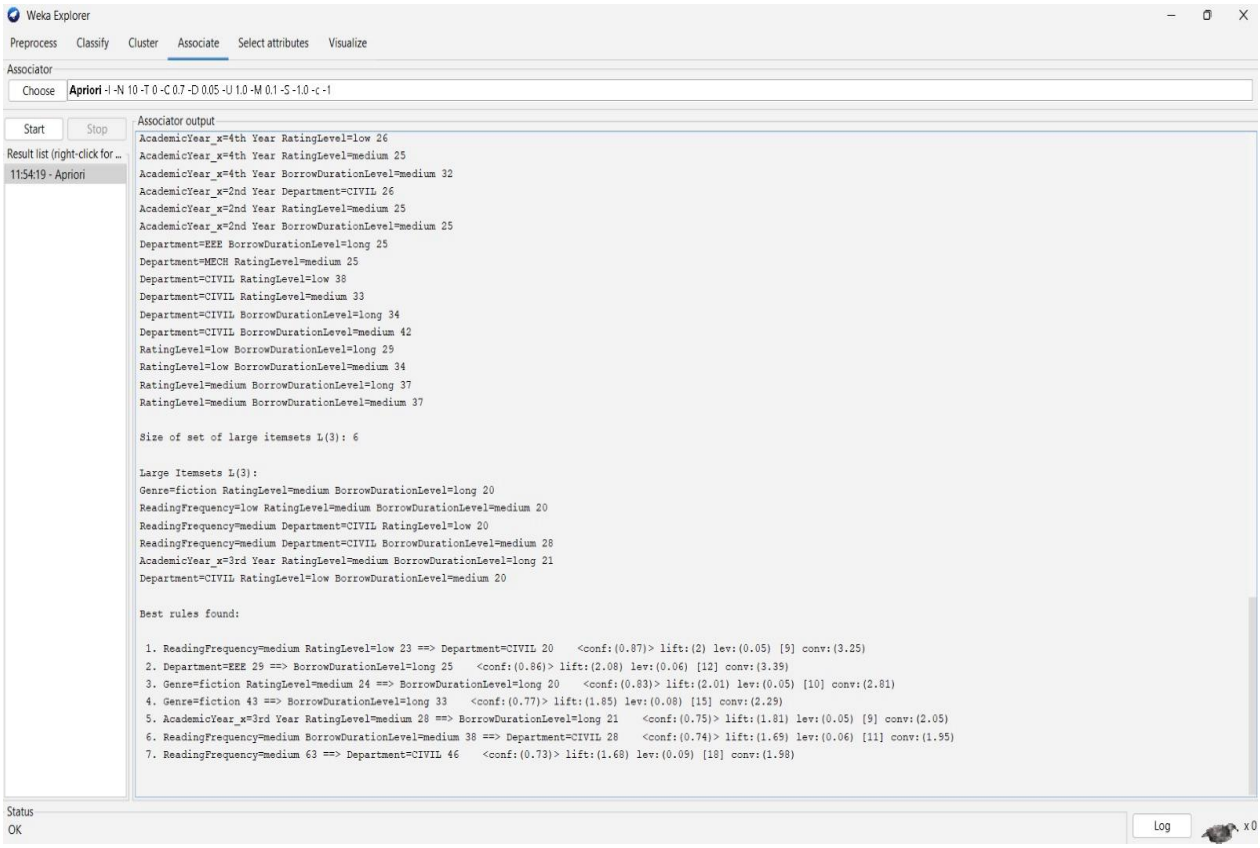

Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Classifier
Choose
RandomTree -K 0 -M 10.0 -V 0.001 -S 1 -depth 5
Test options
Use training set
Supplied test set
Set...
Cross-validation
Folds
10
Percentage split
%
66
More options...
(Nom) RatingLevel
Start
Stop
Result list (right-click for options)
11:30:01 - trees.RandomTree
Classifier output
Size of the tree : 36
Max depth of tree: 5
Time taken to build model: 0.01 seconds
=== Evaluation on training set ===
Time taken to test model on training data: 0.02 seconds
=== Summary ===
Correctly Classified Instances 142 71 %
Incorrectly Classified Instances 58 29 %
Kappa statistic 0.532
Mean absolute error 0.252
Root mean squared error 0.3549
Relative absolute error 60.3678 %
Root relative squared error 77.734 %
Total Number of Instances 200
=== Detailed Accuracy By Class ===
TP Rate FP Rate Precision Recall F-Measure MCC ROC Area PRC Area Class
0.649 0.151 0.716 0.649 0.681 0.509 0.851 0.756 low
0.758 0.284 0.690 0.758 0.723 0.472 0.820 0.785 medium
0.714 0.048 0.758 0.714 0.735 0.682 0.957 0.803 high
Weighted Avg. 0.710 0.194 0.712 0.710 0.709 0.522 0.856 0.777
=== Confusion Matrix ===
a b c <-- classified as
48 23 3 | a = low
17 69 5 | b = medium
2 8 25 | c = high
Status
OK
Log
x 0


Weka Classifier Tree Visualizer: 11:30:01 - trees.RandomTree (final-weka.filters.unsupervised.attribute.Remove-R1-3,5,7-21,23,25)
Tree View
1: Genre
= ml
= non-fiction
= academic
= fiction
= ds
= ai
2: ReadingFrequency
6: Department
12: ReadingFrequency
26: Department
32: ReadingFr
36 : medium (18/8)
= low
= high
= medium
13: Department
19 : high (18/7)
20: Departm
35 : high (9/1)
= EEE
= MECH
= CSE
= CIVIL
= ECE
= EEE
= MECH
= CSE
= CIVIL
= ECE

Association Rule

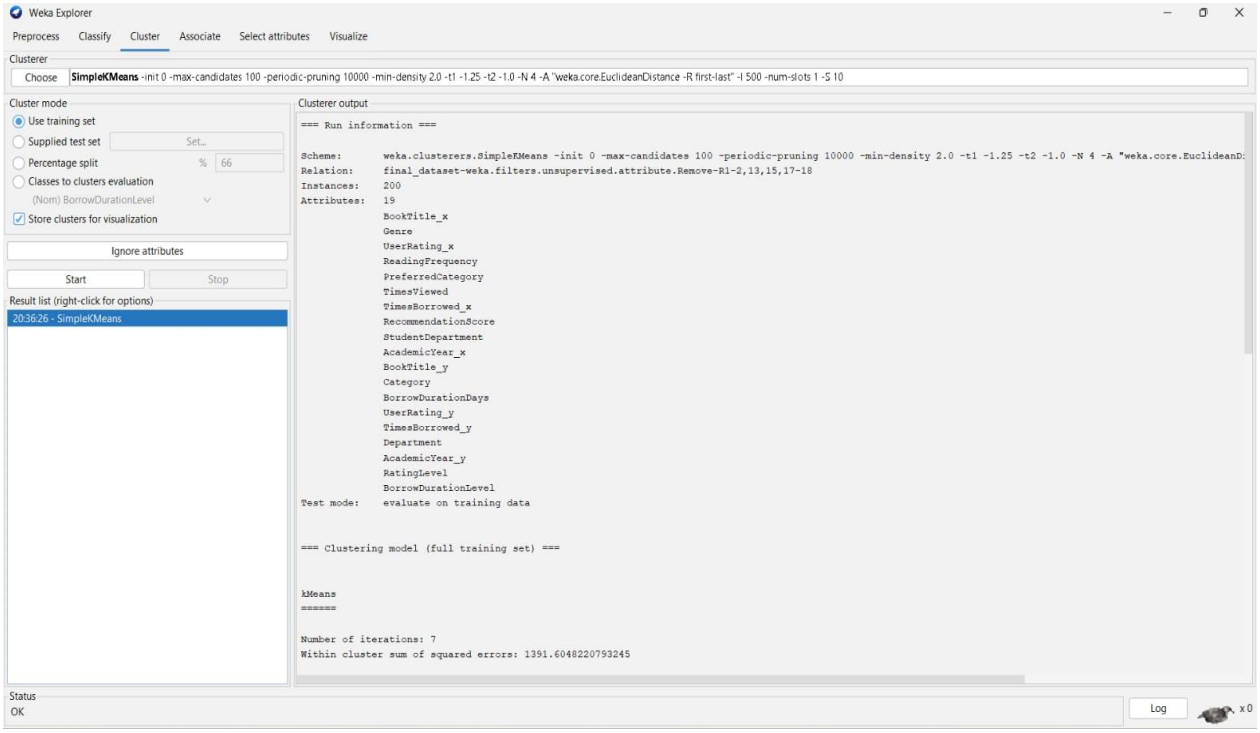
Apriori Algorithm

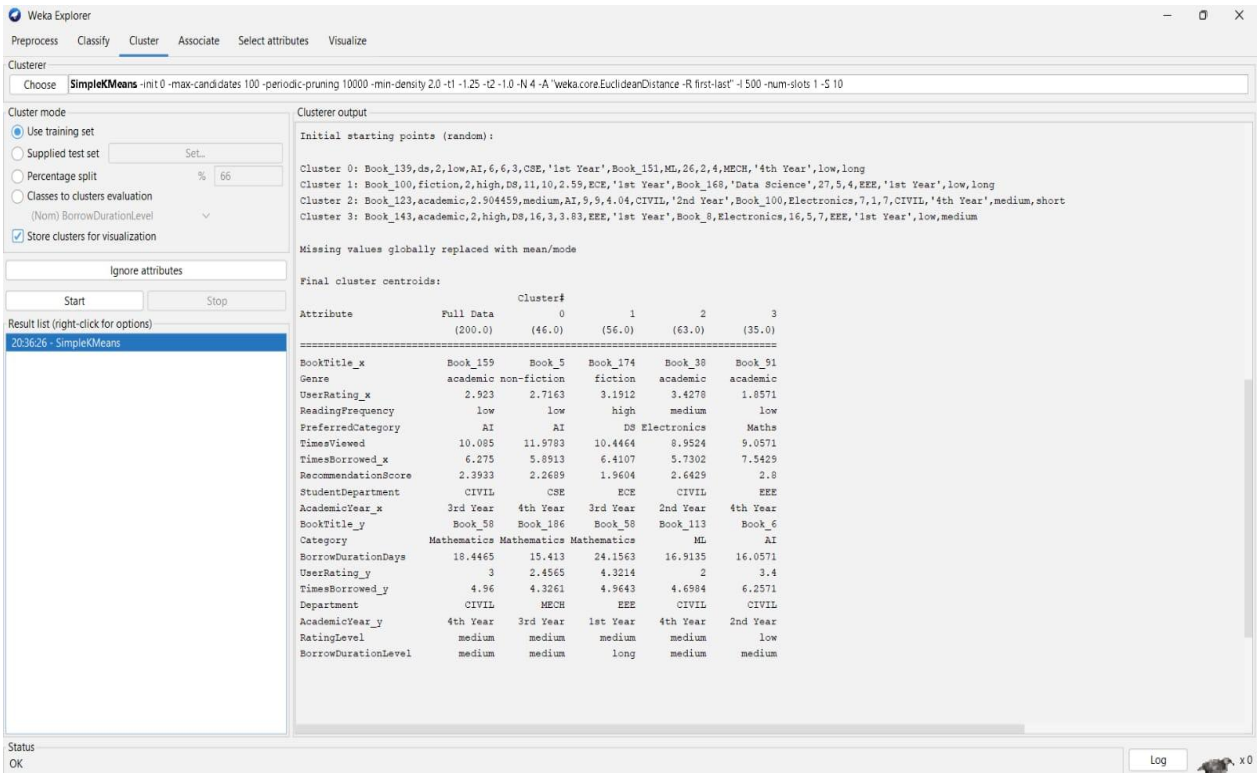


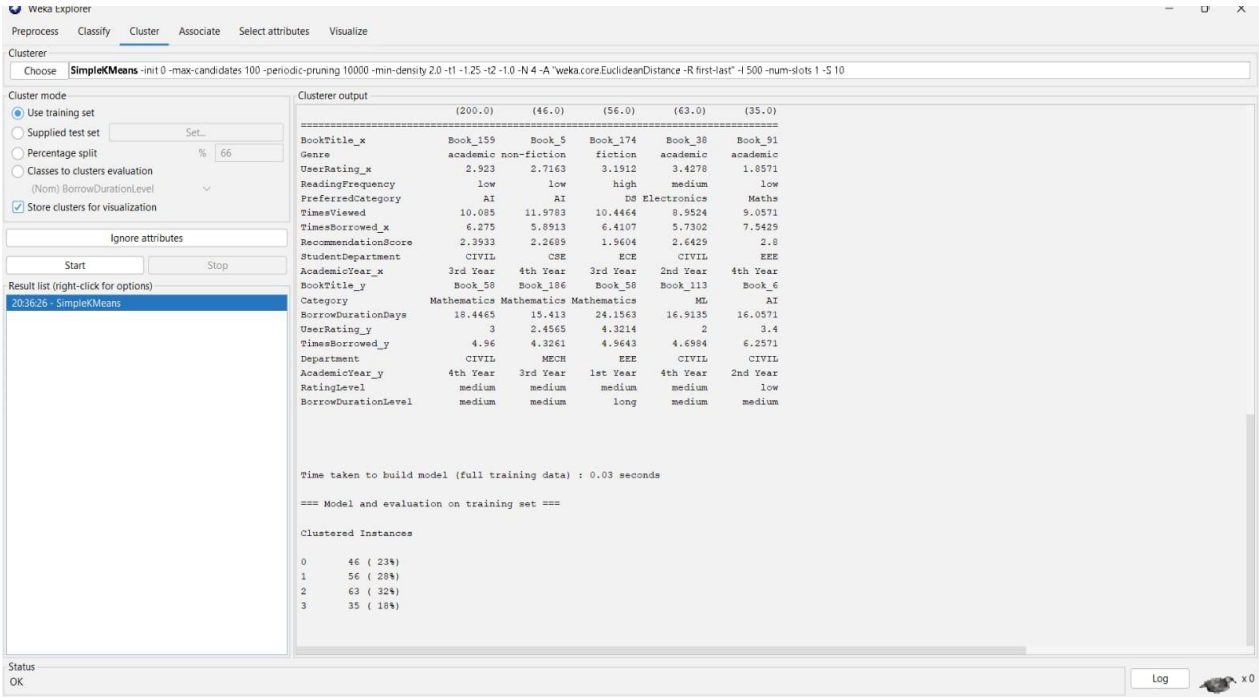


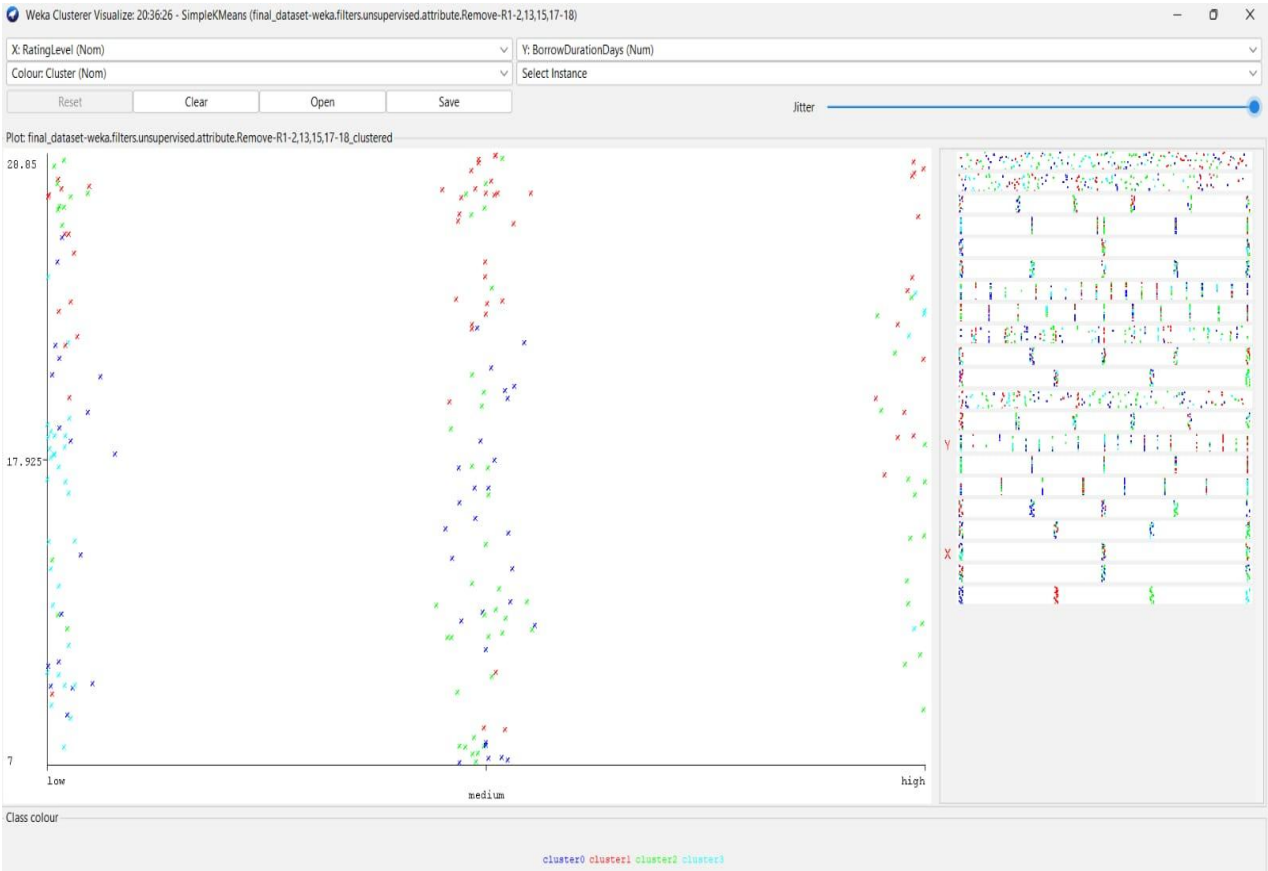
Clustering

Simple KMeans Algorithm






Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Clusterer
Choose
SimpleKMeans -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 4 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10
Cluster mode
Use training set
Supplied test set
Set...
Percentage split
% 66
Classes to clusters evaluation
(Nom) BorrowDurationLevel
Store clusters for visualization
Ignore attributes
Start
Stop
Result list (right-click for options)
20:36:26 - SimpleKMeans
Clusterer output
(200.0) (46.0) (56.0) (63.0) (35.0)
BookTitle_x Book_159 Book_5 Book_174 Book_38 Book_91
Genre academic non-fiction fiction academic academic
UserRating_x 2.923 2.7163 3.1912 3.4278 1.8571
ReadingFrequency low low high medium low
PreferredCategory AI AI DS Electronics Maths
TimesViewed 10.085 11.9783 10.4464 8.9524 9.0571
TimesBorrowed_x 6.275 5.8913 6.4107 5.7302 7.5429
RecommendationScore 2.3933 2.2689 1.9604 2.6429 2.8
StudentDepartment CIVIL CSE ECE CIVIL EEE
AcademicYear_x 3rd Year 4th Year 3rd Year 2nd Year 4th Year
BookTitle_y Book_58 Book_186 Book_58 Book_113 Book_6
Category Mathematics Mathematics Mathematics ML AI
BorrowDurationDays 18.4465 15.413 24.1563 16.9135 16.0571
UserRating_y 3 2.4565 4.3214 2 3.4
TimesBorrowed_y 4.96 4.3261 4.9643 4.6984 6.2571
Department CIVIL MECH EEE CIVIL CIVIL
AcademicYear_y 4th Year 3rd Year 1st Year 4th Year 2nd Year
RatingLevel medium medium medium medium low
BorrowDurationLevel medium medium long medium medium
Time taken to build model (full training data) : 0.03 seconds
=== Model and evaluation on training set ===
Clustered Instances
0 46 (23%)
1 56 (28%)
2 63 (32%)
3 35 (18%)
Status
OK
Log
x 0


Weka Clusterer Visualize: 20:36:26 - SimpleKMeans (final_dataset-weka.filters.unsupervised.attribute.Remove-R1-2,13,15,17-18)
X: RatingLevel (Nom)
Y: BorrowDurationDays (Num)
Colour: Cluster (Nom)
Select Instance
Reset
Clear
Open
Save
Jitter
Plot: final_dataset-weka.filters.unsupervised.attribute.Remove-R1-2,13,15,17-18_clustered
28.85
17.925
7
low
medium
high
Y
X
Class colour
cluster0 cluster1 cluster2 cluster3

8. CONCLUSION

This project successfully applies data mining techniques such as classification, clustering, and association rule mining to analyse user preferences and improve book recommendation systems. The system reduces manual effort, improves efficiency, and provides accurate recommendations. It also helps in understanding user behaviour and identifying patterns in data. The project provides practical knowledge of data analysis and machine learning techniques.

9. FUTURE ENHANCEMENTS

- Development of real-time recommendation systems
- Integration with web applications
- Use of advanced machine learning algorithms
- Improvement in user interface design
- Adding user authentication system

10. REFERENCES

- Python Official Documentation
- Data Mining textbooks
- Research papers on recommendation systems
- Online tutorials and websites

Signatures of Students:

M. A. V. Supriya(24B11AI239)

M. Padmavathi 24B11AI276)

N. S. S. Devi (24B11AI281)

P. Varshitha Devi(24B11AI327)

Signature of Faculty